INF712 - Computerunterstützte Statistik

26-27 WS

08/10/2026

Inhaltsverzeichnis

1 Vorlesung 01-02: Einführung in R **1**

1.1 Grundlagen von R: 1

1.2 Der statistische Werkzeugkasten 15

1.3 Das Minimum, das Sie ohne Hilfe beherrschen sollten 27

1 Vorlesung 01-02: Einführung in R

1.1 Grundlagen von R:

Objekte, Vektoren, Data Frames, Kontrollstrukturen, Funktionen, Grafiken.

R, RStudio, Quarto

R ist die Sprache und RStudio ist die Entwicklungsumgebung, in der die meisten sie schreiben. Quarto (`.qmd`) ist das Dokumentformat, das Fließtext und ausführbaren Code mischt.

Objekte und Zuweisung

R speichert Dinge in *Objekten*. Erzeugt wird ein Objekt mit dem Zuweisungspfeil `<-` (`=` funktioniert auch, der Pfeil ist aber üblich).

```
x <- 5
y <- 3
x + y
#> [1] 8
```

Namen unterscheiden Groß- und Kleinschreibung: `Daten` und `daten` sind zwei verschiedene Objekte. Man sollte Namen bestehender Funktionen vermeiden (`c`, `mean`, `df`, `data`) — es funktioniert, führt aber später zu Verwirrung.

`print()` gibt ein Objekt aus. In der Konsole gibt R automatisch aus, sodass man `print()` selten tippt — innerhalb von Schleifen und Funktionen ist die automatische Ausgabe jedoch abgeschaltet, dort brauchen man es.

```
print(x)
#> [1] 5
cat("Der Wert von x ist", x, "\n")   # cat() gibt ohne Anführungszeichen aus
#> Der Wert von x ist 5
```

`ls()` listet die erzeugten Objekte auf, `rm()` löscht eines, `rm(list = ls())` leert den Arbeitsbereich.

Hilfe bekommen

```
?mean              # Hilfeseite zu einer Funktion
help("mean")       # dasselbe
example(mean)      # die Beispiele der Hilfeseite ausführen
args(mean)         # nur die Argumentliste
??"linear model"   # Volltextsuche über alle Hilfeseiten
```

Jede Hilfeseite ist gleich aufgebaut: **Usage**, **Arguments**, **Value** (was die Funktion zurückgibt) und ganz unten **Examples**. Wenn eine Funktion Sie überrascht, lesen Sie zuerst **Value**.

Vektoren: `c()`

Der Vektor ist das Grundobjekt in R. Eine einzelne Zahl ist lediglich ein Vektor der Länge eins. `c()` („combine", verbinden) erzeugt einen Vektor:

```
alter    <- c(21, 23, 19, 25, 22)
namen    <- c("Ali", "Ahmet", "Can", "Mehmet", "Emre")
bestehen <- c(TRUE, TRUE, FALSE, TRUE, TRUE)

alter
#> [1] 21 23 19 25 22
length(alter)
#> [1] 5
```

Alle Elemente eines Vektors müssen denselben Typ haben. Mischt man sie, wandelt R stillschweigend in den allgemeineren Typ um:

```r
c(1, 2, "drei")      # alles wird Text (character)
#> [1] "1"    "2"    "drei"
c(TRUE, FALSE, 2)    # TRUE wird zu 1, FALSE zu 0
#> [1] 1 0 2
```

Vektoren schnell erzeugen

```r
1:10                           # ganzzahlige Folge
#>  [1]  1  2  3  4  5  6  7  8  9 10
seq(0, 1, by = 0.25)           # Folge mit Schrittweite
#> [1] 0.00 0.25 0.50 0.75 1.00
seq(0, 1, length.out = 5)      # Folge mit fester Anzahl von Punkten
#> [1] 0.00 0.25 0.50 0.75 1.00
rep(c("a", "b"), times = 3)    # den ganzen Vektor wiederholen
#> [1] "a" "b" "a" "b" "a" "b"
rep(c("a", "b"), each = 3)     # jedes Element wiederholen
#> [1] "a" "a" "a" "b" "b" "b"
```

Rechnen erfolgt elementweise

Operationen wirken auf jedes Element einzeln — eine Schleife ist nicht nötig:

```r
alter * 2
#> [1] 42 46 38 50 44
alter - mean(alter)
#> [1] -1  1 -3  3  0
sqrt(alter)
#> [1] 4.582576 4.795832 4.358899 5.000000 4.690416
```

Haben zwei Vektoren unterschiedliche Längen, wird der kürzere *recycelt* (wiederholt). Das ist gelegentlich nützlich und häufig die Ursache eines stillen Fehlers.

Indizierung

Eckige Klammern wählen Elemente aus. **R zählt ab 1**, nicht ab 0.

```r
alter[1]             # erstes Element
#> [1] 21
alter[c(1, 3)]       # erstes und drittes
#> [1] 21 19
alter[-1]            # alles außer dem ersten
#> [1] 23 19 25 22
alter[alter > 21]    # logische Indizierung: die wichtige Form
```

```
#> [1] 23 25 22
namen[alter > 21]    # Auswahl aus einem Vektor mittels Bedingung an einen anderen
#> [1] "Ahmet"  "Mehmet" "Emre"
```

Die logische Indizierung ist das Arbeitspferd der Datenanalyse in R. Lesen Sie `alter > 21` einzeln, dann wird klar, warum das funktioniert:

```
alter > 21
#> [1] FALSE  TRUE FALSE  TRUE  TRUE
```

Namen

Vektorelemente können Namen tragen — so ist der Großteil der statistischen Ausgaben beschriftet:

```
punkte <- c(Ali = 88, Ahmet = 92, Can = 75)
punkte["Ahmet"]
#> Ahmet
#>    92
names(punkte)
#> [1] "Ali"   "Ahmet" "Can"
```

Datentypen und Typumwandlung

Typ	Beispiel	Prüfen	Umwandeln
numerisch (double)	`3.14`	`is.numeric()`	`as.numeric()`
ganzzahlig (integer)	`3L`	`is.integer()`	`as.integer()`
Text (character)	`"Katze"`	`is.character()`	`as.character()`
logisch (logical)	`TRUE`	`is.logical()`	`as.logical()`
Faktor	`factor("niedrig")`	`is.factor()`	`as.factor()`

```
class(alter)
#> [1] "numeric"
class(namen)
#> [1] "character"
str(alter)      # kompakte Struktur: Typ, Länge, erste Werte
#>  num [1:5] 21 23 19 25 22
```

Faktoren — kategoriale Merkmale

Ein **Faktor** ist die Art, wie R ein kategoriales (nominales oder ordinales) Merkmal darstellt. Er sieht aus wie Text, wird aber als ganzzahliger Code plus einer Menge von *Stufen* (`levels`) gespeichert. Nahezu jede statistische Funktion dieser Veranstaltung behandelt Faktoren anders als Text- oder numerische Vektoren — hier korrekt zu arbeiten ist entscheidend.

```
gruppe <- factor(c("Kontrolle", "Behandlung", "Behandlung", "Kontrolle", "Behandlung"))
gruppe
#> [1] Kontrolle  Behandlung Behandlung Kontrolle  Behandlung
#> Levels: Behandlung Kontrolle
levels(gruppe)
#> [1] "Behandlung" "Kontrolle"
table(gruppe)
#> gruppe
#> Behandlung  Kontrolle
#>          3          2
```

Geordnete Faktoren behalten die Reihenfolge ihrer Stufen (relevant für ordinale Daten und für Kontraste):

```
dosis <- factor(c("niedrig", "hoch", "mittel"),
                levels = c("niedrig", "mittel", "hoch"),
                ordered = TRUE)
dosis
#> [1] niedrig hoch    mittel
#> Levels: niedrig < mittel < hoch
```

> ⚠ Der klassische Anfängerfehler
>
> Sind Ihre Gruppen als Zahlen kodiert (1 = Kontrolle, 2 = Behandlung), sieht R ein *numerisches* Merkmal und legt bereitwillig eine Regressionsgerade durch Ihre Gruppencodes. Vorher umwandeln: `gruppe <- factor(gruppe, labels = c("Kontrolle", "Behandlung"))`.

`relevel()` legt fest, welche Stufe die Referenzkategorie ist — wichtig für die Dummy-Kodierung in der Regression:

```
gruppe <- relevel(gruppe, ref = "Kontrolle")
levels(gruppe)
#> [1] "Kontrolle"  "Behandlung"
```

Fehlende Werte

`NA` bedeutet „fehlend" (Not Available / Not Applicable). Der Wert ist ansteckend: Fast jede Rechnung mit `NA` ergibt `NA`. Das ist gewollt — es verhindert, dass man fehlende Daten stillschweigend übergeht.

```
v <- c(4, 7, NA, 2)
mean(v)
#> [1] NA
mean(v, na.rm = TRUE)   # das Argument, das Sie tausendmal tippen werden
#> [1] 4.333333
is.na(v)
#> [1] FALSE FALSE  TRUE FALSE
sum(is.na(v))           # wie viele fehlen
#> [1] 1
complete.cases(v)       # welche Fälle sind vollständig
#> [1]  TRUE  TRUE FALSE  TRUE
```

Verwechseln Sie `NA` (fehlend), `NULL` (gar nichts), `NaN` (Not a Number, wie 0/0) und `Inf` (unendlich, Infinity) nicht.

Matrizen und Listen

Eine **Matrix** ist ein rechteckiger Block eines einzigen Typs.

```
M <- matrix(1:6, nrow = 2)
M
#>      [,1] [,2] [,3]
#> [1,]    1    3    5
#> [2,]    2    4    6
dim(M)
#> [1] 2 3
t(M)          # transponieren
#>      [,1] [,2]
#> [1,]    1    2
#> [2,]    3    4
#> [3,]    5    6
M %*% t(M)    # Matrixmultiplikation
#>      [,1] [,2]
#> [1,]   35   44
#> [2,]   44   56
```

Eine **Liste** kann alles enthalten, auch Objekte unterschiedlichen Typs und unterschiedlicher Länge. Das ist wichtig, denn *die Ausgabe fast jedes statistischen Tests in R ist eine Liste.* Wenn `t.test()` einen hübschen Absatz ausgibt, ist das nur die Druckmethode; darunter liegt eine Liste, in der Sie gezielt nachschauen können.

```r
person <- list(name = "Ali", alter = 21, punkte = c(88, 91))
person$name
#> [1] "Ali"
person[["punkte"]]
#> [1] 88 91
str(person)
#> List of 3
#>  $ name  : chr "Ali"
#>  $ alter : num 21
#>  $ punkte: num [1:2] 88 91
```

Data Frames

Ein Data Frame ist eine Liste gleich langer Vektoren, dargestellt als Tabelle: **Zeilen sind Beobachtungen, Spalten sind Variablen**. In diesem Format werden die Daten während der gesamten Veranstaltung vorliegen.

```r
studierende <- data.frame(
  id      = 1:6,
  name    = c("Ali", "Ahmet", "Can", "Mehmet", "Emre", "Burak"),
  gruppe  = factor(c("A", "B", "A", "B", "A", "B")),
  punkte  = c(88, 92, 75, 84, 79, 95),
  stunden = c(12, 15, 8, 11, 9, 16)
)
studierende
#>   id   name gruppe punkte stunden
#> 1  1    Ali      A     88      12
#> 2  2  Ahmet      B     92      15
#> 3  3    Can      A     75       8
#> 4  4 Mehmet      B     84      11
#> 5  5   Emre      A     79       9
#> 6  6  Burak      B     95      16
```

> **stringsAsFactors**
>
> Seit R 4.0.0 belässt `data.frame()` Textspalten als Text. Ältere Lehrbücher — darunter Field (2012) — gehen vom Gegenteil aus. Beschwert sich eine Funktion, dass sie einen Faktor erwartet, wandeln Sie die Spalte selbst mit `factor()` um.

Einen Data Frame inspizieren

```r
str(studierende)      # Struktur jeder Spalte - führen Sie das immer zuerst aus
#> 'data.frame':    6 obs. of  5 variables:
#>  $ id     : int  1 2 3 4 5 6
#>  $ name   : chr  "Ali" "Ahmet" "Can" "Mehmet" ...
#>  $ gruppe : Factor w/ 2 levels "A","B": 1 2 1 2 1 2
#>  $ punkte : num  88 92 75 84 79 95
#>  $ stunden: num  12 15 8 11 9 16
head(studierende, 3)  # erste Zeilen
#>   id  name gruppe punkte stunden
#> 1  1   Ali      A     88      12
#> 2  2 Ahmet      B     92      15
#> 3  3   Can      A     75       8
nrow(studierende); ncol(studierende); dim(studierende)
#> [1] 6
#> [1] 5
#> [1] 6 5
names(studierende)    # Spaltennamen
#> [1] "id"      "name"    "gruppe"  "punkte"  "stunden"
summary(studierende)  # Kennwerte je Spalte
#>        id              name   gruppe     punkte         stunden
#>  Min.   :1.00   Length   :6   A:3    Min.   :75.00   Min.   : 8.00
#>  1st Qu.:2.25   N.unique :6   B:3    1st Qu.:80.25   1st Qu.: 9.50
#>  Median :3.50   N.blank  :0          Median :86.00   Median :11.50
#>  Mean   :3.50   Min.nchar:3          Mean   :85.50   Mean   :11.83
#>  3rd Qu.:4.75   Max.nchar:6          3rd Qu.:91.00   3rd Qu.:14.25
#>  Max.   :6.00                        Max.   :95.00   Max.   :16.00
```

Auswählen

```r
studierende$punkte                # eine Spalte, als Vektor
#> [1] 88 92 75 84 79 95
studierende[, "punkte"]           # dasselbe
#> [1] 88 92 75 84 79 95
studierende[2, ]                  # zweite Zeile, alle Spalten
#>   id  name gruppe punkte stunden
#> 2  2 Ahmet      B     92      15
studierende[studierende$gruppe == "A", ]              # Zeilen nach Bedingung
#>   id name gruppe punkte stunden
#> 1  1  Ali      A     88      12
#> 3  3  Can      A     75       8
#> 5  5 Emre      A     79       9
studierende[studierende$punkte > 80, c("name", "punkte")]
```

```
#>     name punkte
#> 1    Ali     88
#> 2  Ahmet     92
#> 4 Mehmet     84
#> 6  Burak     95
subset(studierende, punkte > 80, select = c(name, punkte))   # lesbarere Variante
#>     name punkte
#> 1    Ali     88
#> 2  Ahmet     92
#> 4 Mehmet     84
#> 6  Burak     95
```

Ergänzen und ändern

```
studierende$bestanden <- studierende$punkte >= 80
studierende$punkte_z  <- scale(studierende$punkte)[, 1]   # standardisierte Werte
head(studierende, 3)
#>   id  name gruppe punkte stunden bestanden   punkte_z
#> 1  1   Ali      A     88      12      TRUE  0.3263029
#> 2  2 Ahmet      B     92      15      TRUE  0.8483876
#> 3  3   Can      A     75       8     FALSE -1.3704723
```

Sortieren und zusammenfügen

```
studierende[order(-studierende$punkte), c("name", "punkte")]   # absteigend
#>     name punkte
#> 6  Burak     95
#> 2  Ahmet     92
#> 1    Ali     88
#> 4 Mehmet     84
#> 5   Emre     79
#> 3    Can     75
rbind(studierende[1, ], studierende[2, ])                      # Zeilen stapeln
#>   id  name gruppe punkte stunden bestanden  punkte_z
#> 1  1   Ali      A     88      12      TRUE 0.3263029
#> 2  2 Ahmet      B     92      15      TRUE 0.8483876
# merge(df1, df2, by = "id")                                   # über Schlüssel verbinden
```

Daten einlesen und schreiben

```r
dat <- read.csv("daten/experiment.csv")                  # kommagetrennt
dat <- read.csv2("daten/experiment.csv")                 # deutsches Format: ; und ,
dat <- read.table("daten/experiment.dat", header = TRUE, sep = "\t")
dat <- readxl::read_excel("daten/experiment.xlsx")       # Excel
dat <- haven::read_sav("daten/experiment.sav")           # SPSS (Fields Datensätze)

write.csv(studierende, "ergebnisse.csv", row.names = FALSE)
saveRDS(studierende, "studierende.rds")                  # erhält Faktoren und Klassen
studierende <- readRDS("studierende.rds")
```

`getwd()` zeigt, wo R gerade sucht, `setwd()` ändert das Verzeichnis — besser noch: Arbeiten Sie in einem RStudio-**Projekt**, dann bleiben Pfade relativ und übertragbar.

Breit- und Langformat

Viele Funktionen (Messwiederholungs-Grafiken, `ggplot2`, Friedman-Test) erwarten das *Langformat*: eine Zeile pro Beobachtung statt einer Zeile pro Person.

```r
lang <- reshape(breit, direction = "long",
                varying = c("zeit1", "zeit2", "zeit3"),
                v.names = "wert", timevar = "zeit", idvar = "id")

# tidyverse-Entsprechung
library(tidyr)
lang <- pivot_longer(breit, cols = starts_with("zeit"),
                     names_to = "zeit", values_to = "wert")
```

Kontrollstrukturen

`if` / `else`

```r
punktzahl <- 76
if (punktzahl >= 80) {
  print("sehr gut")
} else if (punktzahl >= 60) {
  print("bestanden")
} else {
  print("nicht bestanden")
}
#> [1] "bestanden"
```

`if` verarbeitet **einen** Wert. Für einen ganzen Vektor nehmen Sie `ifelse()`, das elementweise arbeitet:

```r
ifelse(studierende$punkte >= 80, "bestanden", "nicht bestanden")
#> [1] "bestanden"       "bestanden"       "nicht bestanden" "bestanden"
#> [5] "nicht bestanden" "bestanden"
```

for-Schleifen

```r
for (i in 1:5) {
  print(i^2)
}
#> [1] 1
#> [1] 4
#> [1] 9
#> [1] 16
#> [1] 25
```

Man kann über die Elemente jedes Vektors laufen, nicht nur über Zahlen:

```r
for (g in levels(studierende$gruppe)) {
  cat("Gruppe", g, "- mittlere Punktzahl:",
      mean(studierende$punkte[studierende$gruppe == g]), "\n")
}
#> Gruppe A - mittlere Punktzahl: 80.66667
#> Gruppe B - mittlere Punktzahl: 90.33333
```

Ein Objekt innerhalb einer Schleife wachsen zu lassen ist langsam. Legen Sie den Speicher vorher an:

```r
ergebnis <- numeric(10)              # leerer Container passender Größe
for (i in 1:10) ergebnis[i] <- i^2
ergebnis
#>  [1]   1   4   9  16  25  36  49  64  81 100
```

while und repeat

```r
n <- 1
while (n < 100) n <- n * 3
n
#> [1] 243
```

Schleifen, die man meist vermeiden kann

Die `apply`-Familie ersetzt die meisten einfachen Schleifen und ist besser lesbar:

```r
sapply(studierende[, c("punkte", "stunden")], mean)     # Funktion je Spalte
#>   punkte  stunden
#> 85.50000 11.83333
lapply(1:3, function(i) i^2)                            # gibt eine Liste zurück
#> [[1]]
#> [1] 1
#>
#> [[2]]
#> [1] 4
#>
#> [[3]]
#> [1] 9
tapply(studierende$punkte, studierende$gruppe, mean)    # nach Faktor aufteilen
#>        A        B
#> 80.66667 90.33333
aggregate(punkte ~ gruppe, data = studierende, FUN = mean)   # dasselbe als Data Frame
#>   gruppe   punkte
#> 1      A 80.66667
#> 2      B 90.33333
apply(M, 1, sum)                                        # 1 = Zeilen, 2 = Spalten
#> [1]  9 12
replicate(3, mean(rnorm(10)))                           # Ausdruck n-mal wiederholen
#> [1] -0.5900899 -0.4242080 -0.1543617
```

Eigene Funktionen schreiben

```r
se <- function(x, na.rm = TRUE) {
  n <- sum(!is.na(x))
  sd(x, na.rm = na.rm) / sqrt(n)
}

se(studierende$punkte)
#> [1] 3.127832
```

Aufbau:

- `function(...)` deklariert die **Argumente**; `na.rm = TRUE` gibt einem davon einen **Standardwert**, sodass Aufrufende es weglassen dürfen.
- Der Funktionsrumpf steht in `{ }`.
- Der **zuletzt ausgewertete Ausdruck** ist der Rückgabewert. `return()` ist optional und dient vor allem dem vorzeitigen Aussteigen.

- Objekte, die in der Funktion entstehen, sind lokal — sie erscheinen nicht in Ihrem Arbeitsbereich.

Ein etwas größeres Beispiel, das mehrere Werte als Liste zurückgibt:

```
beschreibe <- function(x) {
  x <- x[!is.na(x)]
  list(n = length(x), mittelwert = mean(x), sd = sd(x),
       se = sd(x) / sqrt(length(x)), median = median(x))
}

str(beschreibe(studierende$punkte))
#> List of 5
#>  $ n         : int 6
#>  $ mittelwert: num 85.5
#>  $ sd        : num 7.66
#>  $ se        : num 3.13
#>  $ median    : num 86
```

Man kan eine definierte Funktione mit `replicate()` mehrmals wiederholen.

Pakete

Basis-R bringt viel mit. Alles Weitere kommt aus Paketen: **einmal** installieren, in **jeder** Sitzung laden.

```
install.packages("ggplot2")   # einmal pro Rechner
library(ggplot2)              # in jeder Sitzung, in jedem Skript
```

Die in dieser Veranstaltung verwendeten Pakete, grob nach Wochen:

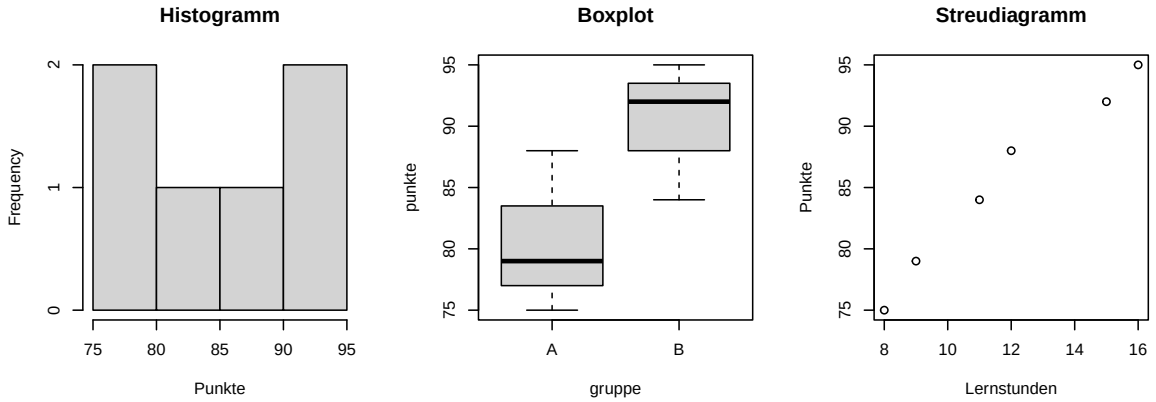
Paket	Wofür	Wochen
`ggplot2`	sämtliche Grafiken bei Field	1–14
`psych`	deskriptive Statistik, Faktorenanalyse	3, 14
`car`	Levene-Test, VIF, Typ-III-ANOVA	4, 7–10
`pastecs`	Kennwerttabellen mit `stat.desc()`	3–5
`ggm` / `ppcor`	Partialkorrelation	6
`multcomp`	Post-hoc-Kontraste	9–10
`pgirmess` / `PMCMRplus`	nichtparametrische Post-hoc-Tests	11
`gmodels` / `vcd`	Kreuztabellen, Zusammenhangsmaße	12
`GPArotation`	Faktorrotation	14

`paket::funktion()` ruft eine Funktion auf, ohne das ganze Paket zu laden — nützlich, wenn zwei Pakete denselben Namen definieren (`car::recode` gegenüber `dplyr::recode`).

Grafiken: Basis-R und `ggplot2`

Basis-R — schnell, schlicht, ideal zur eigenen Kontrolle:

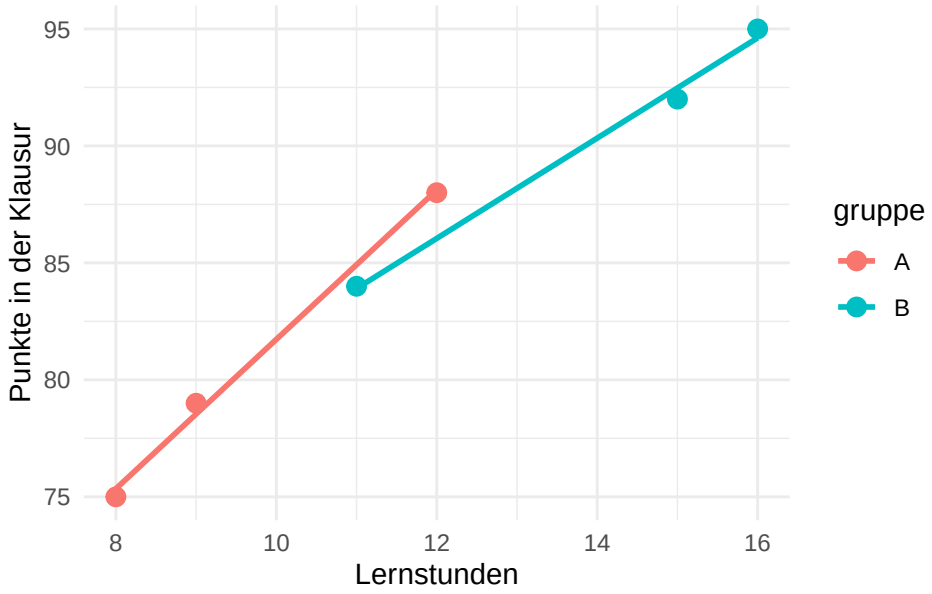
```
par(mfrow = c(1, 3))
hist(studierende$punkte, main = "Histogramm", xlab = "Punkte")
boxplot(punkte ~ gruppe, data = studierende, main = "Boxplot")
plot(studierende$stunden, studierende$punkte, main = "Streudiagramm",
     xlab = "Lernstunden", ylab = "Punkte")
```




```
par(mfrow = c(1, 1))
```

`ggplot2` — schichtweise aufgebaut, ausführlicher, publikationsreif. Das Muster ist stets *Daten* + *Ästhetiken* + *Geome*:

```
library(ggplot2)
ggplot(studierende, aes(x = stunden, y = punkte, colour = gruppe)) +
  geom_point(size = 3) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(x = "Lernstunden", y = "Punkte in der Klausur") +
  theme_minimal()
```



Geome, die Sie brauchen werden: `geom_histogram()`, `geom_density()`, `geom_boxplot()`, `geom_point()`, `geom_bar(stat = "summary")`, `geom_errorbar()`, `geom_qq()`, `geom_smooth()`, dazu `facet_wrap()`, um eine Grafik nach einem Faktor aufzuteilen.

Reproduzierbarkeit

```
set.seed(42)      # legt den Zufallszahlenstrom fest
rnorm(3)
#> [1]  1.3709584 -0.5646982  0.3631284
set.seed(42)
rnorm(3)          # identisch
#> [1]  1.3709584 -0.5646982  0.3631284
```

Setzen Sie vor Simulationen immer einen Startwert (seed). Ohne ihn ändern sich Ihre Monte-Carlo-Ergebnisse bei jedem Durchlauf, und niemand — Sie eingeschlossen — kann sie nachprüfen. `sessionInfo()` dokumentiert Ihre R- und Paketversionen; hängen Sie die Ausgabe an eingereichte Berichte an.

1.2 Der statistische Werkzeugkasten

Fast alle Funktionen geben eine **Liste** zurück. Wenn Sman eine bestimmte Zahl braucht, kann man mit `str()` hineinschauen.

```r
erg <- t.test(punkte ~ gruppe, data = studierende)
erg             # gedruckte Zusammenfassung
erg$p.value     # nur der p-Wert
erg$conf.int    # nur das Konfidenzintervall
str(erg)        # alles, was darin steckt
```

Zufallszahlen, Stichproben, Monte Carlo

Funktion	Was sie tut
`set.seed(n)`	Legt den Zufallsstrom fest, damit Ergebnisse reproduzierbar sind
`runif(n, min, max)`	n Ziehungen aus einer Gleichverteilung — Grundlage der Inversionsmethode
`rnorm(n, mean, sd)`	n Ziehungen aus einer Normalverteilung
`sample(x, size, replace)`	Zufallsstichprobe aus einem Vektor; `replace = TRUE` für Ziehen mit Zurücklegen
`replicate(n, expr)`	Wiederholt einen Ausdruck n-mal und sammelt die Ergebnisse — der Monte-Carlo-Motor
`mean()`, `sd()`, `quantile()`	Fassen die simulierte Verteilung zusammen
`cumsum()`, `cumprod()`	Laufende Summen, nützlich für Konvergenzgrafiken
`system.time(expr)`	Misst die Laufzeit eines Codeblocks

```r
# Inversionsmethode: Exponentialverteilung aus Gleichverteilung (Baron §5.2)
set.seed(1)
u <- runif(10000)
x <- -log(1 - u) / 0.5           # Rate = 0.5
hist(x, breaks = 40, freq = FALSE)
curve(dexp(x, rate = 0.5), add = TRUE, col = "red", lwd = 2)

# Monte-Carlo-Schätzung einer Wahrscheinlichkeit
set.seed(1)
schaetzung <- replicate(10000, {
  wuerfe <- sample(1:6, 2, replace = TRUE)
  sum(wuerfe) >= 10
})
mean(schaetzung)                 # geschätzte Wahrscheinlichkeit
```

Datentypen und Wahrscheinlichkeitsverteilungen

R benennt die Verteilungsfunktionen nach dem Schema **Präfix + Verteilungskürzel**. Wer die vier Präfixe kennt, kennt die ganze Familie:

Präfix	Bedeutung	Beispiel
`d`	**Dichte** bzw. Wahrscheinlichkeit an der Stelle x	`dbinom(3, 10, 0.5)`
`p`	**Verteilungsfunktion**, P(X q)	`pnorm(1.96)`
`q`	**Quantil** — Umkehrung von `p`	`qnorm(0.975)`
`r`	**Zufallszahlen**	`rnorm(100)`

Verteilung	Kürzel	diskret/stetig
Binomialverteilung	`binom`	diskret
Poisson-Verteilung	`pois`	diskret
Geometrische Verteilung	`geom`	diskret
Negative Binomialverteilung	`nbinom`	diskret
Hypergeometrische Verteilung	`hyper`	diskret
Gleichverteilung	`unif`	stetig
Normalverteilung	`norm`	stetig
Exponentialverteilung	`exp`	stetig
Gammaverteilung	`gamma`	stetig
t-Verteilung	`t`	stetig
Chi-Quadrat-Verteilung	`chisq`	stetig
F-Verteilung	`f`	stetig

```r
dbinom(3, size = 10, prob = 0.5)   # P(X = 3)
#> [1] 0.1171875
pnorm(1.96)                        # P(Z <= 1.96)
#> [1] 0.9750021
qnorm(0.975)                       # das 97,5-%-Quantil: 1,96
#> [1] 1.959964
qt(0.975, df = 24)                 # kritischer t-Wert
#> [1] 2.063899
```

Die *Art* des Merkmals beschreiben (Skalenniveau):

`factor()`, `levels()`, `as.numeric()`, `table()`, `cut()` (ein numerisches Merkmal in Klassen einteilen), `curve()` (Dichte über ein Histogramm zeichnen), `hist(..., freq = FALSE)`.

Das **Galton-Brett** demonstriert den zentralen Grenzwertsatz in einer Zeile: viele Bernoulli-Ziehungen aufsummieren und die Form betrachten.

```
set.seed(7)
kugeln <- replicate(5000, sum(sample(c(-1, 1), 12, replace = TRUE)))
hist(kugeln, breaks = 13, freq = FALSE, main = "Galton-Brett")
curve(dnorm(x, mean(kugeln), sd(kugeln)), add = TRUE, col = "red", lwd = 2)
```

Kennwerte, Stichprobenverteilungen, Konfidenzintervalle

Funktion	Was sie tut
`mean()`, `median()`	Lagemaße
`var()`, `sd()`	Streuung (mit Nenner $n - 1$)
`range()`, `IQR()`, `quantile()`	Streuung, robust
`min()`, `max()`, `sum()`, `length()`	Bausteine
`summary()`	Fünf-Punkte-Zusammenfassung plus Mittelwert
`scale()`	Zentriert und standardisiert (z-Werte)
`psych::describe()`	n, Mittelwert, sd, Median, Schiefe, Wölbung, Standardfehler in einer Tabelle
`pastecs::stat.desc(x, basic, norm = TRUE)`	Fields bevorzugte Kennwerttabelle, mit Normalitätsmaßen
`table()`, `prop.table()`	Häufigkeiten und Anteile
`tapply()`, `aggregate()`, `by()`	Dasselbe, nach Gruppen getrennt
`t.test(x)$conf.int`	Konfidenzintervall für einen Mittelwert
`qt()`, `qnorm()`	Kritische Werte, wenn Sie das KI von Hand aufbauen

```
mean(studierende$punkte); sd(studierende$punkte)
#> [1] 85.5
#> [1] 7.661593
se_wert <- sd(studierende$punkte) / sqrt(nrow(studierende))
mean(studierende$punkte) +
  c(-1, 1) * qt(0.975, df = nrow(studierende) - 1) * se_wert
#> [1] 77.45965 93.54035
t.test(studierende$punkte)$conf.int    # dasselbe Intervall, fertig berechnet
#> [1] 77.45965 93.54035
#> attr(,"conf.level")
#> [1] 0.95
```

Parametrische Tests und Voraussetzungsprüfung

Tests

Funktion	Test
`t.test(x, mu = 0)`	Einstichproben-t-Test
`t.test(x, y)`	t-Test für unabhängige Stichproben (standardmäßig nach Welch)
`t.test(y ~ gruppe, data = d)`	Dasselbe in Formelschreibweise
`t.test(x, y, paired = TRUE)`	t-Test für abhängige (verbundene) Stichproben
`t.test(..., var.equal = TRUE)`	Variante mit gepoolter Varianz (Student)
`var.test(x, y)`	F-Test zum Vergleich zweier Varianzen (Baron §9.5)
`prop.test()`, `binom.test()`	Inferenz über Anteile
`power.t.test()`	Teststärke bzw. nötiger Stichprobenumfang

Das Argument `alternative` (`"two.sided"`, `"less"`, `"greater"`) liefert einseitige Tests; `conf.level` ändert die Breite des Konfidenzintervalls.

Voraussetzungen

Funktion	Prüft
`shapiro.test(x)`	Normalverteilung (Shapiro-Wilk)
`qqnorm(x); qqline(x)`	Normalverteilung, grafisch — verlässlicher als der Test
`car::leveneTest(y ~ gruppe, data)`	Varianzhomogenität
`bartlett.test(y ~ gruppe, data)`	Varianzhomogenität, setzt Normalverteilung voraus
`ks.test(x, "pnorm", mean, sd)`	Kolmogorow-Smirnow-Anpassungstest

Effektstärke

```r
library(effsize)
cohen.d(punkte ~ gruppe, data = studierende)     # Cohens d
```

Berichten Sie zu jedem Test eine Effektstärke. Der p-Wert sagt Ihnen *ob*, d sagt Ihnen *wie stark*.

Nichtparametrische Tests

Funktion	Test	Parametrisches Gegenstück
`binom.test()`	Vorzeichentest	Einstichproben-t
`wilcox.test(x, mu = 0)`	Wilcoxon-Vorzeichen-Rang-Test, eine Stichprobe	Einstichproben-t
`wilcox.test(x, y, paired = TRUE)`	Wilcoxon-Vorzeichen-Rang-Test, verbunden	verbundener t-Test
`wilcox.test(y ~ gruppe, data)`	Mann-Whitney-U- / Wilcoxon-Rangsummentest	t-Test für unabhängige Stichproben
`ks.test(x, y)`	Zweistichproben-Kolmogorow-Smirnow-Test	—
`rank()`	Die Ränge, auf denen die Tests beruhen	

Nützliche Argumente: `exact = FALSE` und `correct = FALSE` bei Bindungen, wenn Sie die Zahlen eines Lehrbuchs reproduzieren wollen; `conf.int = TRUE` für eine Schätzung der Lageverschiebung.

Effektstärke aus einem Wilcoxon-Test (nach Field): $r = Z/\sqrt{N}$, wobei sich Z über `qnorm(p_wert / 2)` bestimmen lässt.

Korrelation

Funktion	Was sie tut
`cov(x, y)`	Kovarianz
`cor(x, y)`	Korrelationskoeffizient, ohne Test
`cor(df, use = "complete.obs")`	Vollständige Korrelationsmatrix
`cor(x, y, method = "spearman")`	Spearmans ; `"kendall"` für
`cor.test(x, y)`	Korrelation **mit** p-Wert und Konfidenzintervall
`Hmisc::rcorr(as.matrix(df))`	Matrix der Korrelationen *und* der p-Werte
`ppcor::pcor.test(x, y, z)`	Partialkorrelation, kontrolliert für z
`ggm::pcor()`	Partialkorrelation, Fields Variante
`pairs()` / `GGally::ggpairs()`	Streudiagramm-Matrix

`use = "complete.obs"` (fallweiser Ausschluss) und `use = "pairwise.complete.obs"` steuern den Umgang mit fehlenden Werten — geben Sie an, welche Variante Sie verwendet haben.

```
cor.test(studierende$stunden, studierende$punkte)
#>
#>  Pearson's product-moment correlation
```

```
#>
#> data:  studierende$stunden and studierende$punkte
#> t = 12.06, df = 4, p-value = 0.0002711
#> alternative hypothesis: true correlation is not equal to 0
#> 95 percent confidence interval:
#>  0.8775749 0.9985900
#> sample estimates:
#>       cor
#> 0.9865264
```

Einfache lineare Regression

Funktion	Was sie tut
`lm(y ~ x, data)`	Passt das Modell an; gibt ein Modellobjekt zurück
`summary(modell)`	Koeffizienten, Standardfehler, t, p, R^2, F
`coef(modell)`	Nur die Koeffizienten
`confint(modell)`	Konfidenzintervalle der Koeffizienten
`anova(modell)`	Die ANOVA-Tafel zur Regression
`fitted(modell)`	Vorhergesagte Werte $\hat{y}$
`resid(modell)`	Residuen
`rstandard()`, `rstudent()`	Standardisierte bzw. studentisierte Residuen
`predict(modell, newdata, interval = "confidence")`	Vorhersage mit KI, `"prediction"` für Prognoseintervall
`plot(modell)`	Vier Diagnosegrafiken
`hatvalues()`, `cooks.distance()`, `dfbeta()`	Einflussdiagnostik
`car::durbinWatsonTest(modell)`	Unabhängigkeit der Fehler

```
m <- lm(punkte ~ stunden, data = studierende)
summary(m)
#>
#> Call:
#> lm(formula = punkte ~ stunden, data = studierende)
#>
#> Residuals:
#>       1       2       3       4       5       6
#>  2.1049 -1.0066 -1.4131  0.4754  0.2164 -0.3770
#>
#> Coefficients:
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept)  57.4492     2.3953   23.98 1.79e-05 ***
```

```
#> stunden       2.3705     0.1966   12.06 0.000271 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 1.401 on 4 degrees of freedom
#> Multiple R-squared:  0.9732, Adjusted R-squared:  0.9665
#> F-statistic: 145.4 on 1 and 4 DF,  p-value: 0.0002711
confint(m)
#>                 2.5 %    97.5 %
#> (Intercept) 50.798876 64.099485
#> stunden      1.824761  2.916222
```

Lesen Sie `summary()` in dieser Reihenfolge: die F-Statistik (leistet das Modell überhaupt etwas?), R^2 (wie viel?), dann die Koeffiziententabelle (welche Prädiktoren, wie stark?).

Multiple Regression

Funktion	Was sie tut
`lm(y ~ x1 + x2 + x3, data)`	Mehrere Prädiktoren
`lm(y ~ ., data)`	Alle übrigen Spalten als Prädiktoren
`summary(modell)$adj.r.squared`	Korrigiertes R^2
`anova(modell1, modell2)`	Zusätzliche Quadratsumme / partieller F-Test für geschachtelte Modelle
`update(modell, . ~ . + x4)`	Term hinzufügen oder entfernen
`step(modell, direction = "both")`	Schrittweise Selektion nach AIC
`AIC()`, `BIC()`	Kriterien für den Modellvergleich
`car::vif(modell)`	Varianzinflationsfaktor — Multikollinearität
`1 / car::vif(modell)`	Toleranz
`factor()`, `relevel()`, `contrasts()`	Dummy-Kodierung kategorialer Prädiktoren
`car::outlierTest()`, `influence.measures()`	Fallweise Diagnostik

```
m1 <- lm(punkte ~ stunden, data = studierende)
m2 <- lm(punkte ~ stunden + gruppe, data = studierende)
anova(m1, m2)          # lohnt der zusätzliche Prädiktor?
car::vif(m2)           # > 10 ist ein ernstes Problem; ein Mittel deutlich über 1 mahnt
```

Einfaktorielle Varianzanalyse

Funktion	Was sie tut
`aov(y ~ gruppe, data)`	Passt die ANOVA an; `summary()` liefert die Tafel
`oneway.test(y ~ gruppe, data)`	Welch-F, ohne Annahme gleicher Varianzen
`car::Anova(modell, type = "III")`	Quadratsummen vom Typ II bzw. III
`TukeyHSD(aov_modell)`	Tukey-Post-hoc-Vergleiche
`pairwise.t.test(y, g, p.adjust.method = "bonferroni")`	Paarweise Vergleiche mit Korrektur
`p.adjust()`	Bonferroni-, Holm-, BH-Korrektur von Hand
`contrasts(d$gruppe) <- ...`	Geplante Kontraste (orthogonal, Helmert, polynomial)
`multcomp::glht()`	Allgemeine lineare Hypothesen / eigene Kontraste
`contr.helmert()`, `contr.poly()`, `contr.treatment()`	Eingebaute Kontrastschemata
`car::leveneTest()`	Voraussetzung der Varianzhomogenität
`effectsize::eta_squared()`	Effektstärken ^{2}, ^{2}

```r
fit <- aov(punkte ~ gruppe, data = studierende)
summary(fit)
TukeyHSD(fit)
plot(fit, 1:2)          # Residuen- und Q-Q-Diagramm
```

`aov()` ist `lm()` mit einer ANOVA-typischen Druckmethode — dasselbe Modell, eine andere Tabelle.

Mehrfaktorielle Varianzanalyse

Funktion	Was sie tut
`aov(y ~ A * B, data)`	Beide Haupteffekte **und** die Wechselwirkung
`aov(y ~ A + B, data)`	Nur Haupteffekte
`aov(y ~ A + B + A:B)`	Dasselbe wie `A * B`, ausgeschrieben
`car::Anova(fit, type = "III")`	Typ-III-Quadratsummen — bei unbalancierten Designs
`interaction.plot(A, B, y)`	Die Grafik, die Sie vor jeder Interpretation erstellen müssen
`emmeans::emmeans(fit, ~ A | B)`	Geschätzte Randmittel und einfache Effekte
`emmeans::contrast()`	Tests einfacher Effekte
`effectsize::eta_squared(fit, partial = TRUE)`	Partielles ^{2} je Effekt

```r
fit <- aov(punkte ~ gruppe * dosis, data = d)
summary(fit)
with(d, interaction.plot(dosis, gruppe, punkte))
```

Interpretationsregel: Ist die Wechselwirkung signifikant, interpretieren Sie *zuerst* sie und behandeln die Haupteffekte mit Vorsicht.

Nichtparametrische Varianzanalyse

Funktion	Test	Parametrisches Gegenstück
`kruskal.test(y ~ gruppe, data)`	Kruskal-Wallis-Test	einfaktorielle ANOVA
`friedman.test(as.matrix(df))`	Friedman-Test	ANOVA mit Messwiederholung
`pgirmess::kruskalmc(y, gruppe)`	Post-hoc nach Kruskal-Wallis	
`PMCMRplus::kwAllPairsNemenyiTest()`	Nemenyi-Test Post-hoc-Alternative	
`pairwise.wilcox.test(y, g, p.adjust.method = "holm")`	Paarweise nichtparametrische Vergleiche	
`PMCMRplus::frdAllPairsNemenyiTest()`	Post-hoc nach Friedman	

`friedman.test()` erwartet eine **Matrix** mit Personen in den Zeilen und Bedingungen in den Spalten, keinen Data Frame im Langformat — eine häufige Fehlerquelle.

Analyse kategorialer Daten

Funktion	Was sie tut
`table(a, b)`	Kontingenztafel
`xtabs(~ a + b, data)`	Kontingenztafel aus einer Formel
`prop.table(tab, 1)`	Zeilen- (1) bzw. Spaltenprozente (2)
`addmargins(tab)`	Ergänzt Zeilen- und Spaltensummen
`chisq.test(tab)`	Chi-Quadrat nach Pearson: Unabhängigkeit oder Anpassung
`chisq.test(x, p = probs)`	Anpassungstest gegen eine vorgegebene Verteilung
`chisq.test(tab)$expected`	Erwartete Häufigkeiten — prüfen, ob eine unter 5 liegt
`chisq.test(tab)$residuals`	Standardisierte Residuen: *wo* der Zusammenhang liegt

Funktion	Was sie tut
`fisher.test(tab)`	Exakter Test nach Fisher, bei kleinen erwarteten Häufigkeiten
`chisq.test(tab, correct = TRUE)`	Stetigkeitskorrektur nach Yates (2×2)
`vcd::assocstats(tab)`	Likelihood-Quotient, Phi, Cramérs V
`gmodels::CrossTable(a, b, expected = TRUE)`	Fields Komplett-Kreuztabelle
`MASS::loglm(~ a * b, data = tab)`	Loglineare Analyse
`mosaicplot(tab)`, `barplot(tab)`	Kontingenztafeln grafisch darstellen

```
tab <- table(studierende$gruppe, studierende$bestanden)
tab
#>
#>     FALSE TRUE
#>   A     2    1
#>   B     0    3
chisq.test(tab)
#>
#>  Pearson's Chi-squared test with Yates' continuity correction
#>
#> data:  tab
#> X-squared = 0.75, df = 1, p-value = 0.3865
```

Logistische Regression

Funktion	Was sie tut
`glm(y ~ x, data, family = binomial())`	Passt eine binäre logistische Regression an
`summary(modell)`	Koeffizienten auf der Logit-Skala, z-Tests
`exp(coef(modell))`	Chancenverhältnisse (Odds Ratios) — die interpretierbare Skala
`exp(confint(modell))`	Konfidenzintervalle der Odds Ratios
`predict(modell, type = "response")`	Vorhergesagte Wahrscheinlichkeiten
`predict(modell, type = "link")`	Vorhergesagte Logits
`anova(m1, m2, test = "Chisq")`	Likelihood-Quotienten-Test zwischen geschachtelten Modellen
`modell$deviance`, `modell$null.deviance`	Devianz; die Differenz ist das Modell-Chi-Quadrat
`logLik()`, `AIC()`	Anpassungsmaße; R^2 wird daraus von Hand berechnet (Hosmer-Lemeshow, Cox-Snell, Nagelkerke)

Funktion	Was sie tut
`car::vif(modell)`	Multikollinearität
`nnet::multinom()`	Multinomiale logistische Regression (mehr als zwei Kategorien)

```r
fit <- glm(bestanden ~ stunden + gruppe, data = studierende, family = binomial())
summary(fit)
exp(cbind(OR = coef(fit), confint(fit)))
table(beobachtet = studierende$bestanden,
      vorhergesagt = predict(fit, type = "response") > 0.5)   # Klassifikationstabelle
```

Die abhängige Variable muss ein Faktor oder eine 0/1-Variable sein — prüfen Sie mit `levels()`, welche Kategorie R als „Erfolg" behandelt.

Faktorenanalyse

Funktion	Was sie tut
`cor(df)`	Die Korrelationsmatrix, auf der alles beruht
`psych::cortest.bartlett(df)`	Bartlett-Test auf Sphärizität — ist die Matrix faktorisierbar?
`psych::KMO(df)`	Kaiser-Meyer-Olkin-Maß der Stichprobeneignung
`det(cor(df))`	Determinante; sollte über 0,00001 liegen
`psych::principal(df, nfactors, rotate)`	Hauptkomponentenanalyse, Fields Weg
`psych::fa(df, nfactors, fm = "ml")`	Faktorenanalyse im engeren Sinn
`psych::scree(df)` / `psych::fa.parallel(df)`	Wie viele Faktoren extrahieren?
`factanal(df, factors = k)`	Maximum-Likelihood-Faktorenanalyse in Basis-R
`rotate = "varimax"` / `"oblimin"`	Orthogonale bzw. oblique Rotation (benötigt `GPArotation`)
`print.psych(fit, cut = 0.3, sort = TRUE)`	Lesbare Ladungsmatrix
`psych::alpha(df)`	Cronbachs Alpha für die resultierenden Skalen
`fit$scores`	Faktorwerte je Fall

```r
library(psych); library(GPArotation)
cortest.bartlett(items); KMO(items)
fa.parallel(items, fa = "pc")
```

```r
pc <- principal(items, nfactors = 4, rotate = "oblimin", scores = TRUE)
print.psych(pc, cut = 0.3, sort = TRUE)
alpha(items[, c(1, 4, 7)])
```

1.3 Das Minimum, das Sie ohne Hilfe beherrschen sollten

Sprache — `<-`, `c()`, `seq()`, `rep()`, `length()`, `[ ]`, logische Indizierung, `data.frame()`, `$`, `str()`, `head()`, `summary()`, `subset()`, `factor()`, `levels()`, `is.na()`, `na.rm = TRUE`, `if/else`, `for`, `ifelse()`, `function()`, `library()`, `?`, `set.seed()`, `read.csv()`, `plot()`, `hist()`, `boxplot()`, `ggplot()`.

Statistik — `runif()`, `rnorm()`, `sample()`, `replicate()`, die Präfixe `d/p/q/r`, `mean()`, `sd()`, `var()`, `quantile()`, `tapply()`, `t.test()`, `var.test()`, `shapiro.test()`, `leveneTest()`, `wilcox.test()`, `cor()`, `cor.test()`, `lm()`, `summary()`, `confint()`, `predict()`, `anova(m1, m2)`, `vif()`, `step()`, `aov()`, `TukeyHSD()`, `pairwise.t.test()`, `Anova(type = "III")`, `interaction.plot()`, `kruskal.test()`, `friedman.test()`, `table()`, `chisq.test()`, `fisher.test()`, `glm(family = binomial)`, `exp(coef())`, `principal()` bzw. `fa()`, `KMO()`, `alpha()`.

Sonstige — `help(package = "stats")` — die Liste dessen, was Basis-R alles mitbringt.

Achtung:

1. **Ab 0 zählen.** `x[0]` liefert einen leeren Vektor, nicht das erste Element.
2. **`=` statt `==`** in einer Bedingung. `x = 5` weist zu, `x == 5` vergleicht.
3. **Gruppencodes als Zahlen belassen.** In `factor()` umwandeln, sonst rechnet R eine Regression über die Codes.
4. **`na.rm = TRUE` vergessen** und `NA` als Mittelwert berichten.
5. **Paket nicht geladen.** `Error: could not find function "leveneTest"` heißt, dass `library(car)` fehlt.
6. **Groß-/Kleinschreibung und Tippfehler.** `T.test`, `Daten$Punkte`, `lenght()` — R verzeiht nichts.
7. **Falsches Datenformat.** `friedman.test()` braucht eine Matrix, `ggplot()` meist das Langformat.
8. **Nur den *p*-Wert lesen.** Zuerst die Voraussetzungen prüfen, danach eine Effektstärke berichten.
9. **Kein `set.seed()`** vor einer Simulation — nichts ist reproduzierbar.

10. **Unpaarige Klammern oder Anführungszeichen.** Ein `+` an der Eingabezeile bedeutet, dass R noch wartet — mit `Esc` abbrechen.